

SORBONNE UNIVERSITÉ
Faculté des Sciences et Ingénierie
Laboratoire d'Informatique de Paris 6 (LIP6)

RAPPORT DE STAGE DE MASTER 1

**Décodage contraint pour la génération de données
JSON : limitations des solutions existantes**

Analyse fine des erreurs de conformité de Guidance, Outlines et XGrammar

Réalisé par : Wafaa Berrais
Master 1 MIND (Machine Learning, Intelligence Artificielle et
Données)

Encadrement : Mohamed-Amine Baazizi (LIP6, Sorbonne Université)
Nour El Houda Ben Ali (Université Paris Dauphine-PSL)

Année universitaire 2025–2026

Résumé

Les grands modèles de langue (LLM) sont de plus en plus sollicités pour produire des données JSON conformes à un schéma exprimé en JSON Schema. Le décodage contraint vise à garantir cette conformité : à chaque étape de la génération, il masque les tokens qui violeraient le schéma. Mais JSON Schema est un langage très expressif, et les moteurs existants n'en couvrent qu'imparfaitement la sémantique. Ce stage étudie finement les limitations de trois moteurs largement utilisés, Guidance/LLGuidance, Outlines et XGrammar, à partir du benchmark JSONSchemaBench.

Nous rejouons environ 76 000 décisions portant sur des couples schéma–instance dont la validité est connue. Nous distinguons deux types d'erreurs : l'acceptation d'une instance invalide (UNDER) et le rejet d'une instance valide (OVER). Après une analyse exploratoire, nous construisons plusieurs centaines de caractéristiques décrivant le schéma, l'instance et leurs interactions.

Des modèles d'apprentissage (forêts aléatoires, boosting) montrent d'abord que ces erreurs sont prévisibles sur des schémas jamais vus, avec un F1 compris entre 0,74 et 0,84. Ils servent ensuite à sélectionner les caractéristiques les plus informatives. Des arbres de décision peu profonds, entraînés sur ces caractéristiques, produisent des règles lisibles qui constituent le résultat principal du travail. Ces arbres atteignent un F1 de 0,60 à 0,80 sur des schémas non vus. Celui d'Outlines UNDER, par exemple, détecte 61,5 % de ces erreurs avec une précision de 90,7 %.

Appliqués sans réentraînement à des schémas Kubernetes, les arbres restent très précis mais perdent en rappel pour deux des trois moteurs, signe d'un décalage de distribution.

Enfin, une expérience pilote place un vrai LLM dans la boucle (Qwen2.5-0.5B-Instruct sous XGrammar). Les cas signalés par les règles y échouent bien plus souvent que des cas témoins : 15 sur 15 contre 4 sur 15 pour OVER, 8 sur 15 contre 2 sur 15 pour UNDER. Les règles obtenues ouvrent ainsi la voie à plusieurs usages : cibler l'analyse du code des moteurs, adapter les prompts ou déclencher une validation supplémentaire dans les cas à risque.

Mots-clés : décodage contraint, JSON Schema, grands modèles de langue, sorties structurées, analyse d'erreurs, règles interprétables.

1. Introduction

Les grands modèles de langue ne servent plus seulement à produire du texte libre. Ils sont de plus en plus intégrés dans des chaînes de traitement qui attendent des sorties structurées : appels d’outils par des agents, extraction d’informations, alimentation de bases de données ou d’API. Dans ces usages, la sortie doit respecter un format précis, le plus souvent un document JSON conforme à un schéma exprimé dans le standard JSON Schema. Une sortie non conforme peut interrompre toute la chaîne.

Or un LLM laissé libre ne garantit pas le respect d’un format. Il peut produire un JSON mal formé, omettre un champ obligatoire ou violer une contrainte du schéma. Plusieurs solutions existent pour y remédier, depuis la formulation soignée des prompts jusqu’à la validation a posteriori suivie d’une régénération. Les principaux fournisseurs (OpenAI, Anthropic, Google) proposent aussi des modes de génération structurée, tout en déclarant des limitations sur les mots-clés acceptés et sur la taille des schémas. Parmi ces solutions, le décodage contraint (*constrained decoding*) s’est imposé comme l’approche de référence : le schéma est compilé en une grammaire, et à chaque étape de la génération le moteur interdit les tokens qui mèneraient à une sortie invalide. Plusieurs moteurs open source mettent en œuvre ce principe, notamment Outlines [5], XGrammar [3] et Guidance avec son moteur LLGuidance [4].

Pourtant, même ces moteurs se trompent. JSON Schema permet de combiner des sous-schémas (`allOf`, `anyOf`, `oneOf`, `not`), d’exprimer des conditions (`if/then/else`), de borner des valeurs numériques ou de contraindre des noms de propriétés par des expressions régulières (`patternProperties`). Certaines de ces contraintes se traduisent mal en grammaire, et les moteurs doivent faire des approximations. Ils acceptent alors parfois des sorties invalides, ou au contraire interdisent des sorties valides. Des benchmarks récents comme JSONSchemaBench [1] et SLOT [2] mesurent ces limitations, mais par des scores globaux qui ne disent pas dans quels cas les moteurs échouent ni pourquoi. La question qui guide ce travail est donc la suivante : *dans quels cas les moteurs de décodage contraint se trompent-ils, et quelles sont précisément leurs limitations ?*

Conformément au sujet du stage, notre objectif est d’étudier de manière fine les limitations des solutions de décodage contraint pour la génération de données JSON. Nous partons des benchmarks existants et les étendons par une analyse détaillée des caractéristiques des schémas utilisés. Nous nous concentrons sur les trois moteurs open source cités. Au fil des expériences, cet objectif général s’est précisé : caractériser les configurations de schéma et d’instance qui provoquent des erreurs, puis les résumer sous forme de règles interprétables et exploitables.

La suite du rapport suit cette démarche. La section 2 présente les moteurs, les données et le protocole d’exécution. La section 3 résume l’analyse exploratoire, et la section 4 l’ingénierie des caractéristiques. La section 5 présente la modélisation et les règles extraites. La section 6 confronte ces règles à une génération réelle par un LLM. La section 7 discute les limites et les perspectives, avant de conclure. L’ensemble du code et des résultats est disponible dans le dépôt du projet¹.

¹<https://github.com/wafaaBerrais/constrained-decoding-json-limits>

2. Moteurs étudiés, données et protocole

2.1 Les trois moteurs

Les trois moteurs partagent le même principe mais diffèrent par la manière de représenter le schéma. **Outlines** traduit le schéma en expression régulière, puis en automate indexé sur le vocabulaire du modèle. **XGrammar** le traduit en grammaire hors contexte et s'appuie sur un automate à pile, avec un cache des masques de tokens. **Guidance/LLGuidance** calcule les masques à la volée à l'aide d'un analyseur syntaxique. Ces choix de représentation déterminent directement quelles contraintes de JSON Schema peuvent être exprimées exactement, et lesquelles doivent être approximées.

2.2 Données retenues

Nous utilisons JSONSchemaBench [1], qui rassemble des schémas réels issus de plusieurs sources, et son extension MaskBench. Chaque test associe un schéma à une instance JSON dont la validité est connue. Comme l'exécution des trois moteurs sur l'ensemble du benchmark est coûteuse en temps, nous nous sommes concentrés sur les sous-ensembles **GitHub**, classés par difficulté croissante (*trivial, easy, medium, hard, ultra*). Le sous-ensemble **Kubernetes**, qui provient d'un autre domaine, a été gardé de côté pour tester la généralisation des résultats.

2.3 Protocole d'exécution

Chaque test est rejoué à travers le décodeur contraint de chaque moteur : l'instance est tokenisée, puis validée token par token contre le masque calculé par le moteur. La décision du moteur (instance acceptée ou rejetée) est comparée à la validité attendue, ce qui donne l'une des quatre issues du tableau 1.

Issue	Signification
Correcte	instance valide acceptée, ou instance invalide rejetée
UNDER (sous-contrainte)	instance <i>invalide</i> acceptée : le moteur est trop permissif
OVER (sur-contrainte)	instance <i>valide</i> rejetée : le moteur est trop restrictif
Timeout / crash	la compilation ou la validation n'aboutit pas

Tab. 1 : Issues possibles d'un test.

Le dispositif mesure séparément la durée de chaque étape (compilation de la grammaire, tokenisation, validation). Un délai maximal par schéma, des points de reprise et une supervision des exécutions ont permis de mener les longues campagnes sur un serveur partagé malgré les plantages. Au total, nous avons obtenu environ 76 000 décisions moteur $\times$ test. La figure 1 résume l'ensemble de la démarche.

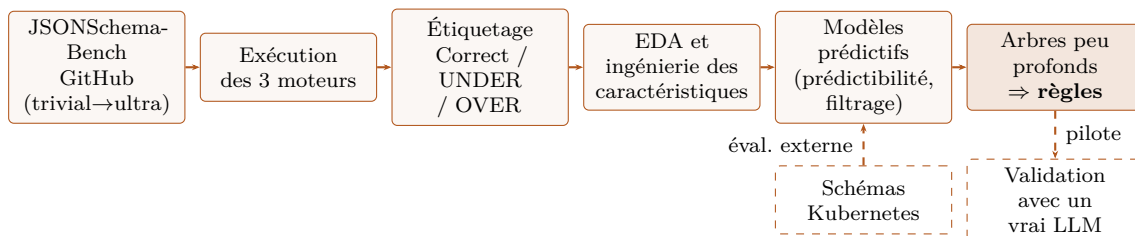
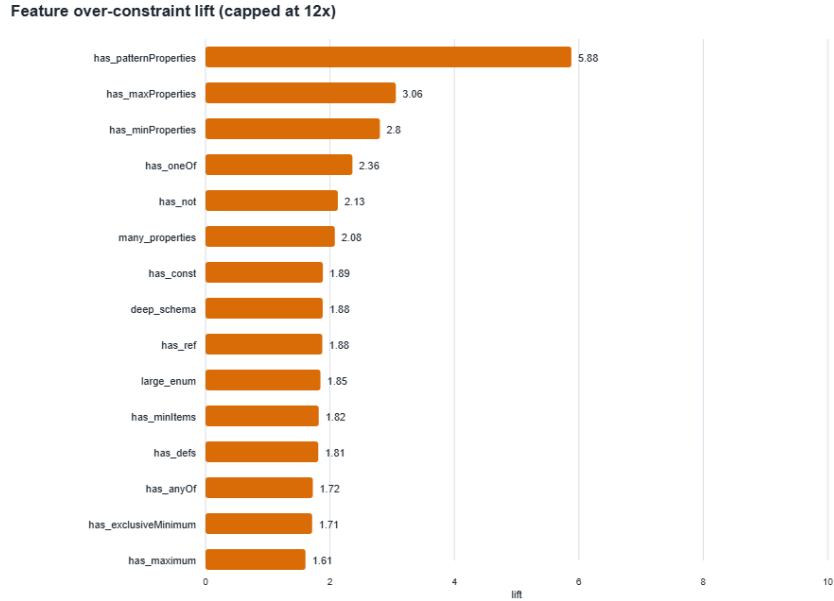


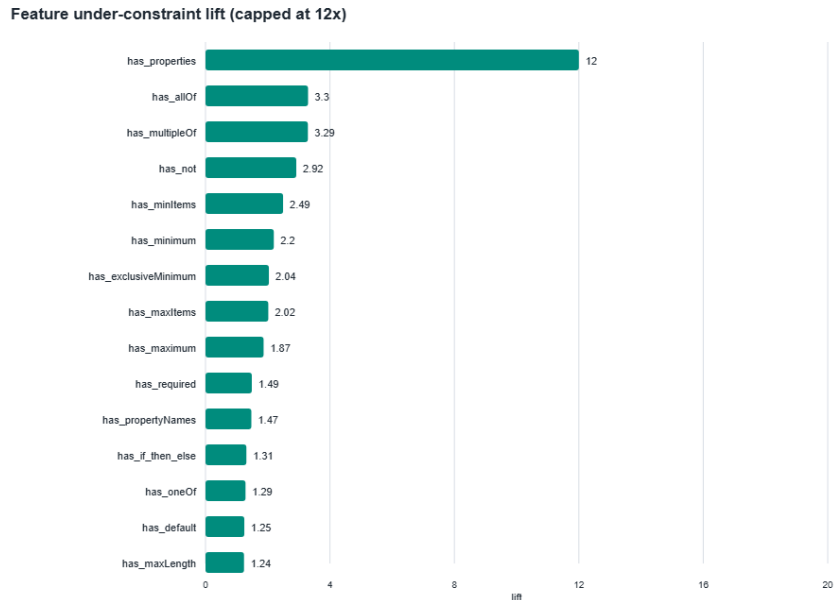
Fig. 1 : Vue d'ensemble de la démarche.

3. Analyse exploratoire

Avant toute modélisation, nous avons étudié la répartition des issues par moteur, par sous-ensemble et par mot-clé de JSON Schema. Pour chaque mot-clé, nous calculons le taux d'UNDER parmi les instances invalides et le taux d'OVER parmi les instances valides des schémas qui l'utilisent. Nous mesurons aussi, pour une caractéristique donnée, son *support* (sa fréquence) et son *lift*, c'est-à-dire le rapport entre le taux d'erreur parmi les tests qui la présentent et le taux d'erreur global. Un lift de 2 signifie ainsi que l'erreur est deux fois plus fréquente en présence de la caractéristique. Ces mesures sont calculées à l'échelle du test et à l'échelle du schéma. La figure 2 présente les lifts obtenus pour XGrammar.



(a) Erreurs OVER (sur-contrainte).



(b) Erreurs UNDER (sous-contrainte).

Fig. 2 : XGrammar : lift des principales caractéristiques de schéma pour chaque type d'erreur (valeurs plafonnées à 12).

Trois constats ressortent de cette analyse. D'abord, les moteurs ont des profils d'erreur très différents : Guidance ne produit aucune erreur UNDER dans nos données et ne se trompe que par

excès de restriction, alors qu'Outlines et XGrammar produisent les deux types d'erreurs.

Ensuite, chez XGrammar, les deux types d'erreurs ne sont pas associés aux mêmes contraintes. Les erreurs OVER sont surtout liées aux contraintes portant sur les propriétés d'un objet : **patternProperties** (lift de 5,88), **maxProperties** (3,06) et **minProperties** (2,80), suivies des combinateurs **oneOf** et **not**. Ces contraintes, qui portent sur les noms ou le nombre de propriétés, sont difficiles à exprimer dans une grammaire, et le moteur semble alors rejeter des instances pourtant valides. L'analyse des taux d'erreur par mot-clé montre aussi que **contains** et **if/then/else** conduisent surtout à des OVER. Les erreurs UNDER, elles, sont fortement associées à la présence de **properties** (lift plafonné à 12 sur la figure), puis à **allOf** (3,30), **multipleOf** (3,29) et **not** (2,92), ainsi qu'aux bornes numériques (**minimum**, **exclusiveMinimum**, **maximum**) et aux bornes sur les tableaux (**minItems**, **maxItems**). Cela suggère que certaines de ces contraintes sont relâchées plutôt qu'appliquées strictement.

Enfin, la seule présence d'un mot-clé ne suffit pas à expliquer les erreurs : un même mot-clé peut être bien géré dans un schéma et mal géré dans un autre, selon le contexte où il apparaît et selon la valeur testée. C'est ce dernier constat qui a motivé l'ingénierie des caractéristiques.

4. Ingénierie des caractéristiques

L'analyse exploratoire reposait sur des indicateurs simples : présence d'un mot-clé (`has_patternProperties`, `has_multipleOf`, etc.) et indicateurs structurels (`many_properties`, `deep_schema`, `large_enum`). Pour capturer le contexte, nous les avons enrichis jusqu'à obtenir plusieurs centaines de caractéristiques, calculées automatiquement pour chaque test. Elles se regroupent en quelques familles :

- **Contraintes numériques** : présence de bornes (`minimum`, `maximum`, bornes exclusives), profondeur d'imbrication de ces contraintes, et position de la valeur testée par rapport aux bornes (sur la borne, à l'intérieur, à l'extérieur).
- **Structure des objets** : usage de `additionalProperties`, nombre de propriétés et propriétés obligatoires.
- **Expressions régulières** : usage de `pattern` et de `patternProperties`, complexité des motifs.
- **Combinateurs et conditions** : usage de `allOf`, `anyOf`, `oneOf`, `not` et `if/then/else`.
- **Instance et complexité globale** : longueur des chaînes de l'instance, taille et profondeur du schéma.
- **Interactions schéma $\times$ instance** : par exemple, une valeur située sur une borne d'une contrainte imbriquée.

Ce jeu initial était volontairement large, pour ne pas exclure trop tôt une piste. Il contenait cependant beaucoup de caractéristiques redondantes ou peu informatives, ce qui rendait les modèles difficiles à interpréter. Nous l'avons donc affiné en deux temps : un filtrage selon l'importance mesurée par les modèles prédictifs (section 5), puis une vérification guidée par la connaissance du domaine, pour ne garder que des caractéristiques ayant un sens clair vis-à-vis de JSON Schema. Chaque caractéristique est documentée dans le dépôt. Parmi les caractéristiques finales figurent par exemple `instance_string_total_length` (longueur totale des chaînes de l'instance), `type_mismatch_count` (incohérences de type), `string_pattern_complexity_score` (complexité des regex), `numeric_boundary_case_below_min` (valeur sous le minimum) ou `instance_matching_pattern_keys_count` (clés de l'instance correspondant à un motif). La figure 3 montre leur poids dans l'un des modèles prédictifs de la section 5.

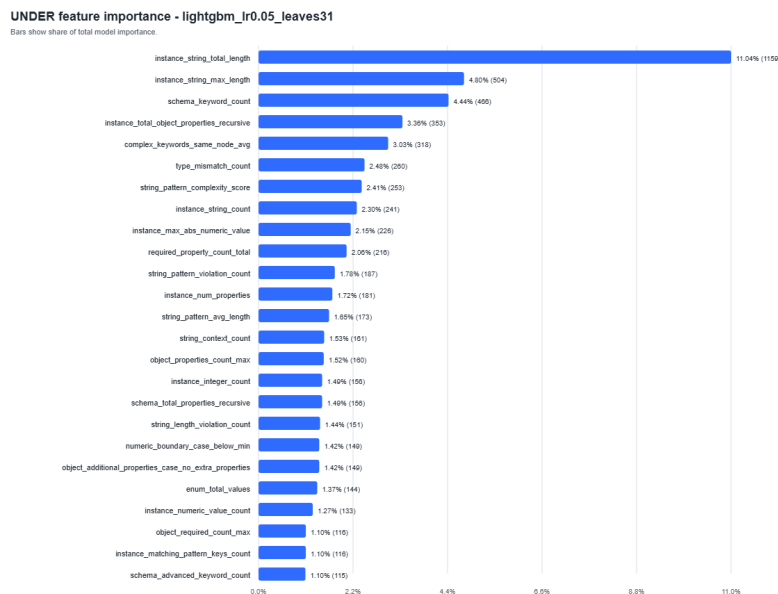


Fig. 3 : Outlines UNDER (LightGBM) : part de chaque caractéristique dans l'importance totale du modèle. La longueur des chaînes de l'instance, la complexité du schéma et les cas limites numériques portent l'essentiel du signal.

5. Modélisation et extraction de règles

5.1 Les erreurs sont-elles prévisibles ?

Avant de chercher des règles, il fallait vérifier que les erreurs ne sont pas aléatoires, c'est-à-dire qu'elles peuvent être prédites à partir des caractéristiques. Nous avons entraîné un classifieur par couple moteur $\times$ type d'erreur, en comparant la régression logistique, les forêts aléatoires, HistGradientBoosting et LightGBM. Point important du protocole : les découpages entraînement / validation / test sont **groupés par schéma**, de sorte que tous les tests d'un même schéma tombent dans le même ensemble. Sans cela, le modèle pourrait reconnaître un schéma déjà vu au lieu d'apprendre des configurations générales. Les scores du tableau 2 portent donc sur des schémas jamais vus à l'entraînement.

Moteur	Erreur	Modèle	Précision	Rappel	F1	ROC-AUC
Outlines	UNDER	LightGBM	0,96	0,75	0,84	0,99
Outlines	OVER	Random Forest	0,89	0,74	0,81	0,90
Guidance	OVER	Random Forest	0,71	0,87	0,78	0,84
XGrammar	UNDER	Random Forest	0,71	0,78	0,74	0,95
XGrammar	OVER	Random Forest	0,78	0,70	0,74	0,93

Tab. 2 : Prédiction des erreurs sur l'ensemble de test (schémas non vus). Guidance ne produisant aucune erreur UNDER, ce cas n'est pas modélisé.

Les cinq modèles atteignent un F1 d'au moins 0,74 et une ROC-AUC comprise entre 0,84 et 0,99. Une validation croisée à 5 plis, également groupée par schéma, donne des ROC-AUC du même ordre (entre 0,84 et 0,96). Les erreurs sont donc largement déterminées par des caractéristiques observables du schéma et de l'instance. Ces modèles n'étaient pas une fin en soi : leur second rôle était d'identifier les caractéristiques les plus utiles (figure 3), afin de fournir aux arbres de décision un jeu réduit et interprétable.

5.2 Des arbres de décision aux règles

Le cœur du travail consiste à transformer ces signaux en règles lisibles. Pour chaque couple moteur $\times$ type d'erreur, nous entraînons un arbre de décision sur les caractéristiques retenues, en le gardant volontairement petit : profondeur maximale de 5 et au moins 50 tests par feuille. Un seuil de décision est fixé pour chaque arbre (0,86 pour Outlines UNDER, par exemple). Chaque chemin menant à une feuille qui prédit une erreur se lit comme une règle de la forme « si telle et telle condition, alors le moteur risque de se tromper ».

Ce sont ces arbres, et non les modèles prédictifs, qui produisent les règles. La crédibilité des règles dépend donc directement de la qualité des arbres, que nous mesurons sur le même ensemble de test groupé par schéma (tableau 3). La précision y indique la part des alertes de l'arbre qui correspondent à de vraies erreurs, et le rappel la part des erreurs qu'il détecte.

Moteur	Erreur	Précision	Rappel	F1 arbre	F1 modèle prédictif
Guidance	OVER	0,746	0,847	0,793	0,78
Outlines	UNDER	0,907	0,615	0,733	0,84
Outlines	OVER	0,867	0,737	0,797	0,81
XGrammar	UNDER	0,450	0,904	0,601	0,74
XGrammar	OVER	0,897	0,642	0,749	0,74

Tab. 3 : Performances des arbres de décision sur l'ensemble de test (schémas non vus), comparées au F1 du meilleur modèle prédictif (tableau 2).

Pour trois couples sur cinq, l'arbre fait presque aussi bien que le meilleur modèle prédictif, voire légèrement mieux : la simplification nécessaire à l'interprétation coûte peu. L'écart est plus

marqué pour les erreurs UNDER, avec un F1 de 0,73 contre 0,84 pour Outlines et de 0,60 contre 0,74 pour XGrammar. L'arbre XGrammar UNDER est le moins fiable : il détecte 90 % des erreurs, mais plus de la moitié de ses alertes sont fausses (précision de 0,45). Ses règles sont donc à lire comme des pistes plutôt que comme des conclusions.

5.3 Ce que disent les règles

Exemple détaillé : Outlines, UNDER. Les branches de l'arbre qui prédisent une erreur se résument à la condition suivante : la contrainte numérique est imbriquée à une profondeur supérieure ou égale à 2, et la valeur testée se trouve sur une borne ou en dehors (hors cas où elle est exactement égale au minimum). Dans ce cas, Outlines accepte l'instance invalide.

Sur l'ensemble de test (1 405 tests, dont 143 erreurs UNDER), l'arbre détecte **88 erreurs** pour seulement **9 fausses alertes** : précision de 90,7 %, rappel de 61,5 %.

Cette règle isole à elle seule la majorité des erreurs UNDER d'Outlines, avec très peu de fausses alertes. Elle oriente vers une hypothèse précise : la traduction du schéma en expression régulière ne conserverait pas correctement les bornes numériques dans les contextes imbriqués. Cette hypothèse reste à confirmer par une analyse du code du moteur, mais la règle indique exactement où chercher. Le tableau 4 résume les signaux qui ressortent des arbres pour chaque couple moteur × type d'erreur.

Moteur, erreur	Principaux signaux dans les règles
Guidance, OVER	nombreuses propriétés déclarées dans le schéma, nombreux contextes de type chaîne, objets complexes, instances contenant de longues chaînes ou des valeurs numériques
Outlines, UNDER	contraintes numériques profondes, valeurs sur une borne ou en dehors, contexte objet sans propriétés supplémentaires
Outlines, OVER	clés de l'instance non reconnues en présence de <code>additionalProperties</code> , <code>allOf</code> , <code>patternProperties</code> , <code>additionalProperties: false</code>
XGrammar, UNDER	<code>allOf</code> partiellement satisfait, incohérence de type ou <code>required</code> à l'intérieur des combineurs
XGrammar, OVER	<code>patternProperties</code> dont les motifs correspondent à des clés de l'instance, en contexte <code>additionalProperties</code>

Tab. 4 : Signaux principaux identifiés par les arbres de décision.

Ces règles recoupent l'analyse exploratoire : chez XGrammar, `patternProperties` ressortait déjà comme le principal facteur d'OVER, et `allOf` et `not` parmi les facteurs d'UNDER. Les arbres précisent toutefois le contexte. Pour XGrammar OVER, ce n'est pas la simple présence de `patternProperties` qui pose problème, mais le cas où des clés de l'instance correspondent effectivement aux motifs. Plus généralement, les erreurs OVER se concentrent sur les contraintes qui portent sur les propriétés des objets (`patternProperties`, `additionalProperties`, nombre de propriétés), tandis que les erreurs UNDER touchent surtout les bornes numériques et les combineurs.

5.4 Généralisation à Kubernetes

Pour vérifier que les règles restent crédibles en dehors des données d'entraînement, nous avons appliqué les arbres entraînés sur GitHub, sans réentraînement, aux schémas Kubernetes (tableau 5). Les modèles prédictifs ont fait l'objet d'une évaluation externe analogue, dont les résultats figurent dans le dépôt ; nous présentons ici celle des arbres, puisque ce sont eux qui portent les règles.

Les règles Guidance OVER se transfèrent bien : leurs performances sur Kubernetes sont presque identiques à celles obtenues sur GitHub. Pour Outlines et XGrammar, les règles restent fiables lorsqu'elles se déclenchent (précision de 0,99 à 1,00), mais elles ne couvrent qu'une partie des

erreurs (rappel de 0,23 à 0,37). Les schémas Kubernetes déclenchent en effet des motifs d'erreur rares dans les données GitHub, ce qui illustre un décalage de distribution. Les erreurs UNDER n'ont pas pu être évaluées sur Kubernetes, car ni XGrammar ni Outlines n'en produisent sur ce sous-ensemble.

Moteur	Erreur	Précision	Rappel	F1
Guidance	OVER	0,707	0,856	0,774
Outlines	OVER	1,000	0,372	0,542
XGrammar	OVER	0,988	0,234	0,379

Tab. 5 : Arbres entraînés sur GitHub appliqués sans réentraînement aux schémas Kubernetes.

6. Validation avec un vrai LLM

Toutes les expériences précédentes rejouent des instances connues à travers le masque de tokens : aucun modèle ne génère quoi que ce soit. Une règle qui signale un risque d'erreur ne dit donc pas encore si ce risque se concrétise lorsqu'un LLM écrit lui-même le JSON. Pour le vérifier, nous avons mené une expérience pilote complémentaire qui place un vrai LLM dans la boucle. Elle poursuit deux objectifs : vérifier qu'une règle qui annonce un risque d'erreur correspond à une défaillance réelle en génération, et tester une correction guidée par les règles. Le code et les résultats de cette expérience se trouvent dans le dossier `llm_validation` du dépôt.

6.1 Protocole

Le modèle Qwen2.5-0.5B-Instruct [6] est exécuté localement avec le décodage contraint de **XGrammar**, sur une *tâche de copie* : le modèle doit renvoyer en JSON une instance connue qui lui est fournie. Cette tâche a l'avantage de fixer exactement la sortie attendue. Les règles ne sont pas montrées au modèle : elles servent d'aiguillage et décident, avant la génération, si un cas est à risque. La notion d'erreur dépend du type étudié :

- **OVER** (instances valides, schémas Kubernetes) : il y a erreur si le modèle ne parvient pas à reproduire l'instance, parce que la grammaire a bloqué une sortie pourtant valide ;
- **UNDER** (instances invalides, schémas GitHub de l'ensemble de test) : il y a erreur si la sortie échoue à la validation par la bibliothèque `jsonschema`, parce que la grammaire a laissé passer une sortie invalide.

Pour chaque type d'erreur, 15 cas signalés par une règle sont comparés à 15 cas témoins que les règles ne signalent pas, soit 60 cas au total.

6.2 Les règles se vérifient en génération réelle

La figure 4 compare la part de cas où la génération échoue, pour les cas signalés et pour les témoins. Les cas signalés échouent bien plus souvent : 15 sur 15 contre 4 sur 15 pour OVER, et 8 sur 15 contre 2 sur 15 pour UNDER. Les erreurs observées chez les témoins sont pour l'essentiel des erreurs de copie du petit modèle, et non des erreurs du moteur. Le signal est particulièrement net pour OVER : dans tous les cas signalés, XGrammar empêche le modèle de produire l'instance valide demandée.

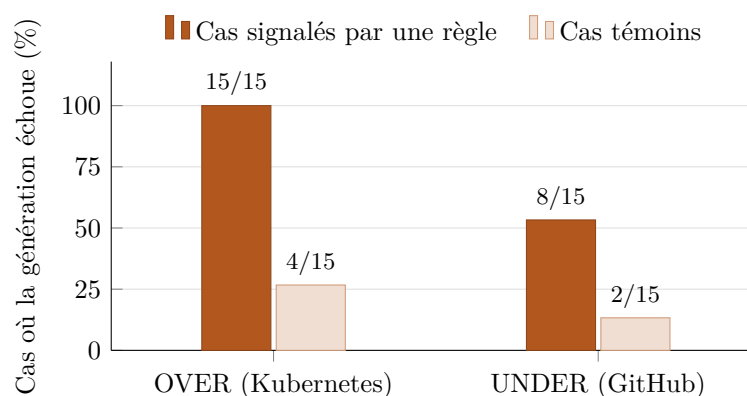


Fig. 4 : Génération réelle avec Qwen2.5-0.5B-Instruct sous XGrammar : part des cas où la génération échoue, pour les cas signalés par une règle et pour des cas témoins non signalés (15 cas par barre).

Exemple : OVER sur un schéma Kubernetes. Le schéma n'interdit pas les propriétés supplémentaires, donc l'instance demandée est valide. Elle contient une clé `"fsType_"` qui se termine par une espace.

Instance demandée	<code>{"pdID": "disk-12345", "fsType_": "ext4"}</code>
Règle déclenchée	l'instance contient une propriété supplémentaire autorisée par le schéma
Sortie sous XGrammar	<code>{"pdID": "disk-12345"} : la clé supplémentaire ne peut pas être générée</code>
Sortie sans grammaire	<code>{"pdID": "disk-12345", "fsType": "ext4"} : valide, mais le modèle a « corrigé » la clé</code>

6.3 Une correction guidée par les règles ?

Puisque les règles signalent les cas à risque avant la génération, on peut adapter la stratégie pour ces seuls cas. Avec un modèle de 0,5 milliard de paramètres, les gains restent limités. Pour les cas à risque OVER, générer sans la grammaire produit une sortie valide vis-à-vis du schéma dans 15 cas sur 15, mais une seule reproduit exactement l'instance (13 sur 15 si l'on ignore les espaces dans les clés). Comme le montre l'exemple ci-dessus, le modèle a tendance à « normaliser » les clés inhabituelles. Pour les cas à risque UNDER, ajouter le schéma dans le prompt amène rarement le modèle à corriger la valeur invalide. En revanche, une validation par `jsonschema` après la génération détecte toutes les sorties invalides restantes, ce qui conforte l'idée d'une validation en plusieurs couches déclenchée par les règles (section 7).

Ce pilote (60 cas, un seul petit modèle, un seul moteur) vise à vérifier la démarche, et non à fournir des chiffres statistiquement solides. Il montre toutefois que des règles apprises sans aucune génération signalent bien des cas où un LLM échoue réellement.

7. Limites et perspectives

Limites. Le stage a duré un mois et demi, et l'exécution des moteurs est coûteuse : nous nous sommes donc limités aux sous-ensembles GitHub, avec Kubernetes comme seul test de généralisation. Certaines erreurs sont rares, ce qui limite ce que les modèles peuvent apprendre ; l'absence d'erreurs UNDER chez Guidance, et chez tous les moteurs sur Kubernetes, empêche par exemple d'étudier ces cas. Les arbres, plus simples que les modèles prédictifs, perdent en performance sur les erreurs UNDER, et l'arbre XGrammar UNDER reste peu précis. Le recul du rappel sur Kubernetes montre aussi que les règles apprises sur un domaine ne couvrent pas forcément toutes les erreurs d'un autre. La validation avec un vrai LLM reste un pilote de 60 cas, mené avec un seul petit modèle et un seul moteur. Par ailleurs, nos résultats dépendent des versions des moteurs testées, qui évoluent vite. Enfin, les règles décrivent des associations entre configurations et erreurs, et non leurs causes : expliquer une erreur demande encore d'examiner le fonctionnement interne du moteur concerné.

Perspectives. Les règles extraites sont conçues pour être réutilisées. Elles permettent d'abord de **restreindre l'investigation** aux configurations à risque, pour comprendre précisément ce qui échoue dans chaque moteur et, à terme, proposer des corrections aux mainteneurs. Elles peuvent aussi servir au **prompt engineering**, en signalant au LLM les combinaisons de contraintes qui demandent une attention particulière. Enfin, elles peuvent déclencher une **validation en plusieurs couches** : lorsqu'un schéma et une sortie correspondent à une règle à risque, une validation supplémentaire avec un validateur JSON Schema complet peut être ajoutée après la génération. Le pilote de la section 6 va dans ce sens : c'est cette validation a posteriori qui intercepte toutes les sorties invalides restantes.

Plusieurs pistes prévues par le sujet n'ont pas pu être traitées faute de temps et restent des perspectives : l'expérimentation des API commerciales (OpenAI, Anthropic, Gemini), l'étude des modèles spécialisés dans la génération de code, et l'extension de l'analyse au benchmark SLOT [2] et aux autres sous-ensembles de JSONSchemaBench. Réentraîner les modèles sur des schémas plus variés permettrait aussi de réduire l'effet du décalage de distribution. Enfin, étendre la validation en génération réelle à des modèles plus grands, aux deux autres moteurs et à davantage de cas permettrait de confirmer ces premiers résultats et de mieux évaluer les corrections guidées par les règles.

8. Conclusion

Ce stage a étudié en détail les limitations de trois moteurs de décodage contraint, Guidance, Outlines et XGrammar, face à JSON Schema. En jouant environ 76 000 décisions sur JSONSchemaBench, nous avons montré que leurs erreurs, qu’elles soient trop permissives ou trop restrictives, ne sont pas aléatoires : elles sont liées à des configurations précises de schéma et d’instance, et peuvent être prédites sur des schémas jamais vus. Nous avons résumé ces configurations sous forme de règles interprétables, comme celle qui capture la majorité des erreurs UNDER d’Outlines autour des bornes numériques imbriquées. Les arbres qui portent ces règles restent proches des meilleurs modèles prédictifs, et ceux de Guidance se transfèrent bien à des schémas d’un autre domaine. Une expérience pilote avec un vrai LLM confirme enfin que les cas signalés par les règles sont bien ceux où la génération échoue. Ces règles constituent un point de départ concret pour comprendre, contourner et corriger les limites des moteurs actuels.

Références

- [1] S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz et H. Nori. *JSON-SchemaBench : A Rigorous Benchmark of Structured Outputs for Language Models*. arXiv :2501.10868, 2025.
- [2] D. Y.-B. Wang, Z. Shen, S. S. Mishra, Z. Xu, Y. Teng et H. Ding. *SLOT : Structuring the Output of Large Language Models*. 2025.
- [3] Y. Dong, C. F. Ruan, Y. Cai, R. Lai, Z. Xu, Y. Zhao et T. Chen. *XGrammar : Flexible and Efficient Structured Generation Engine for Large Language Models*. Proceedings of Machine Learning and Systems 7, 2024.
- [4] M. Moskal, H. Nori, H. Cooper et L. Huynh. *LLGuidance : Making Structured Outputs Go Brrr*. 2025.
- [5] B. T. Willard et R. Louf. *Efficient Guided Generation for Large Language Models*. arXiv :2307.09702, 2023.
- [6] Qwen Team (A. Yang et al.). *Qwen2.5 Technical Report*. arXiv :2412.15115, 2024.